STA258: Statistics with Applied Probability

Introduction to R

Nishan Mudalige and Samir Hamdi

Fall 2025

Copyright Notice

The content of these lecture slides is the intellectual property of the author(s) and is protected by copyright law. Unauthorized distribution, reproduction, or use of this material, in whole or in part, is strictly prohibited. This includes but is not limited to, sharing these slides online, copying, or paraphrasing the content without proper attribution. Any use of this material must be for educational purposes only and with explicit permission from the author(s).

Outline

1 Operations and Assignment

2 Basic Functions

3 Some Data Structures

4 Plots

5 Libraries

6 Reading a File

Basic operations

```
5 + 2
## [1] 7

5 - 2
## [1] 3

5 * 2
## [1] 10

5 / 2
## [1] 3.5

5^2
## 25

5 ** 2
## 25

5 %% 2
## 1
```

Order of operations

```
# The natural order of operations in R is
# Parentheses: (, )
# Exponents: ^ or **
# Multiply: *
# Divide: /
# Add: +
# Subtract: -

7 + 5 - 4 * 9 / 3 ^ 2
## 8

## Change the order of operations with parentheses
7 + (5 - (4 * 9 / 3)) ^ 2
## 56
```

Assigning Variables

```
a <- 7
a
## [1] 7

b = 2
b
## [1] 2

a + b
## [1] 9

c = a + b
c
## [1] 9
```

Outline

1 Operations and Assignment

2 Basic Functions

3 Some Data Structures

4 Plots

5 Libraries

6 Reading a File

Basic Functions

R has with many useful functions available with the base version.

```
# logarithm
log(100)

# exponential
exp(3)

# absolute value
abs(-7.25)

# square root
sqrt(82)

# floor
floor(27.3488)

# ceiling
ceiling(27.3488)
```

The Help Function

We can get help with a function by reading its documentation.

Let's examine the documentation of the `log` function

```
help(log)

?log
```

The `log` function has 2 parameters which are `x` and `base`. Some related functions are also provided in the documentation.

Use this information to find $\log_{10}(100)$.

Outline

1 Operations and Assignment

2 Basic Functions

3 Some Data Structures

4 Plots

5 Libraries

6 Reading a File

Vectors

A collection of the same type of item.

```
## Syntax: c(item_1, item_2, ..., item_n)

c(4, 1, 7, 5, 2, 8, 6)
## [1] 4, 1, 7, 5, 2, 8, 6

x = c(4, 1, 7, 5, 2, 8, 6)
x
## [1] 4, 1, 7, 5, 2, 8, 6


y = c("mon", "tue", "wed", "thu", "fri", "sat", "sun")
y
## [1] "mon" "tue" "wed" "thu" "fri" "sat" "sun"
```

Vectors Ctd ...

Indexing locations in a vector

```
x = c(4, 1, 7, 5, 2, 8, 6)
y = c("mon", "tue", "wed", "thu", "fri", "sat", "sun")

x[1]
## 4

x[2]
## 1

y[1]
[1] "mon"

y[2]
[1] "tue"
```

Vectors Ctd ...

Operations with vectors

```
x = c(4, 1, 7, 5, 2, 8, 6)
y = c("mon", "tue", "wed", "thu", "fri", "sat", "sun")

x + 10
## [1] 14 11 17 15 12 18 16

x^2
## [1] 16  1 49 25  4 64 36

y + 10  ## What happens? Why?
```

Some Functions on Vectors

```
x = c(4, 1, 7, 5, 2, 8, 6)
y = c("mon", "tue", "wed", "thu", "fri", "sat", "sun")

exp(x)
## Function applied to all elements

exp(y)
## Error

z1 = c(1, 2, 3)
z2 = c(1:7)

x + x   ## What happens?

x + z1  ## What happens?

x + z2  ## What happens?
```

Some Functions on Vectors

```
x = c(4, 1, 7, 5, 2, 8, 6)

length(x)

min(x)

max(x)

mean(x)

median(x)

var(x)

sd(x)

## Would these functions work on vector y from the previous slide?
```

Some Functions on Vectors

```
x = c(4, 1, 7, 5, 2, 8, 6)

summary(x)
##    Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
##   1.000   3.000   5.000   4.714   6.500   8.000


z = c(3, 5, 5, 1, 2, 4, 5, 2, 2, 4, 1, 3, 4, 2, 4)
table(z)
## z
## 1 2 3 4 5
## 2 4 2 4 3
```

Dataframes

A `dataframe` is a tabular data structure which allows a column to be of the same data type

	Column 1	**Column 2**	...	**Column** m
Row 1	–	–	...	–
Row 2	–	–	...	–
⋮	⋮	⋮	⋱	⋮
Row n	–	–	...	–

Dataframes Ctd...

```
marks = data.frame(ID = c(1001, 1002, 1003, 1004, 1005, 1006, 1007,
                  1008, 1009, 1010),
                  math = c(92, 85, 22, 95, 90, 87, 76, 48, 76, 76),
                  stats = c(87, 66, 76, 86, 62, 87, 54, 77, 87, 68),
                  campus = c("UTM", "UTSC", "St. George", "UTM", "UTSC",
                  "UTM", "St. George", "St. George", "UTSC", "St. George"))
```

Dataframes Ctd...

```
## Using the data frame created on the previous slide

marks
##       ID math stats      campus
## 1   1001   92    87         UTM
## 2   1002   85    66        UTSC
## 3   1003   22    76 St. George
## 4   1004   95    86         UTM
## 5   1005   90    62        UTSC
## 6   1006   87    87         UTM
## 7   1007   76    54 St. George
## 8   1008   48    77 St. George
## 9   1009   76    87  UTSC, St.
## 10  1010   76    68      George
```

Dataframes Ctd...

```
## Isolate the stats column
marks[, 3]
## [1] 87 66 76 86 62 87 54 77 87 68

marks[, "stats"]
## [1] 87 66 76 86 62 87 54 77 87 68

marks$stats
## [1] 87 66 76 86 62 87 54 77 87 68
```

Dataframes Ctd...

```
## Isolate the ID, math and stats column
marks[, c(1,2, 3)]
## or marks[c("ID", "math", "stats")]

##      ID math stats
## 1  1001   92    87
## 2  1002   85    66
## 3  1003   22    76
## 4  1004   95    86
## 5  1005   90    62
## 6  1006   87    87
## 7  1007   76    54
## 8  1008   48    77
## 9  1009   76    87
## 10 1010   76    68
```

Exercises

1. Find the mean of the math scores.
2. Find the number of students from each campus.

Use the help feature to read up about the `aggregate` and `cbind` functions

3. Find the mean of the math scores by campus.
4. Find the mean of the math scores and stats scores by campus.

Data Sets in R

- Base R has some data sets by default.
- We will use some for examples.
- It's possible to obtain a list of the data sets using `data` .

```
data()


unlist(data())
```

Outline

1 Operations and Assignment

2 Basic Functions

3 Some Data Structures

4 Plots

5 Libraries

6 Reading a File

Plots

- It's often useful to get information on a data set with a data visualization.
- Let's use the `iris` data set in R.

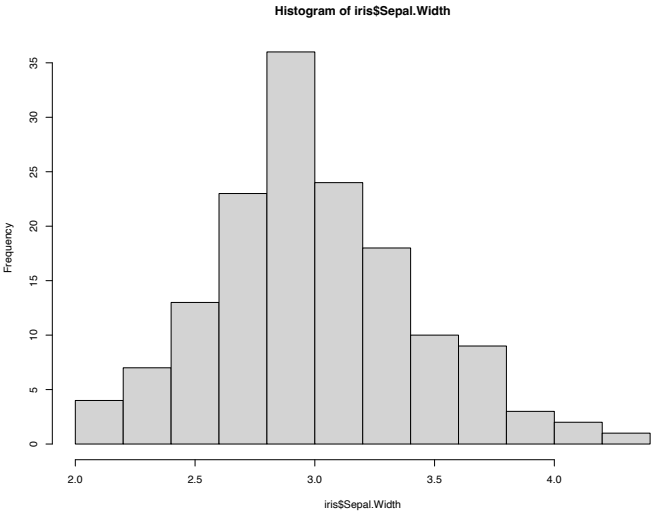
```
help(iris)

names(iris)
## [1] "Sepal.Length" "Sepal.Width"  "Petal.Length" "Petal.Width"
    "Species"

head(iris)
##   Sepal.Length Sepal.Width Petal.Length Petal.Width Species
## 1          5.1         3.5          1.4         0.2  setosa
## 2          4.9         3.0          1.4         0.2  setosa
## 3          4.7         3.2          1.3         0.2  setosa
## 4          4.6         3.1          1.5         0.2  setosa
## 5          5.0         3.6          1.4         0.2  setosa
## 6          5.4         3.9          1.7         0.4  setosa
```

Histogram

- A common graphical representation of quantitative data is a histogram.
- It is a graphical summary which can be prepared for data summarized in either a frequency, relative frequency, or percent frequency distribution.
- The variables of interest is on the horizontal axis and the frequency, relative frequency, or percent frequency on the vertical axis.
- Let's create a histogram of the `Sepal.Width` from the `iris` data set.
- We can get information on the *shape* of the distribution, central tendency and skewness.

```
hist(iris$Sepal.Width)
```

Histogram Ctd...

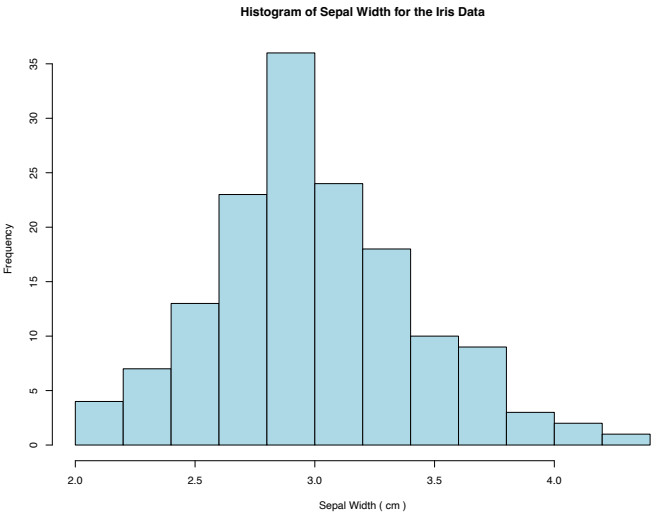


Histogram Ctd...

- We can improve the histogram with a better x-axis label, title and colours.

```
hist(iris$Sepal.Width,
     xlab = "Sepal Width ( cm )",
     main = "Histogram of Sepal Width for the Iris Data",
     col = "lightblue")

## A list of default colours can be found using
## colours()
```

Histogram Ctd...



Exercises

1. Get information on the bins, counts, and mid-points of the histogram for sepal widths.
2. Create a histogram of sepal widths with fewer bins.
3. Create a histogram with more colours for the bars.

Boxplot

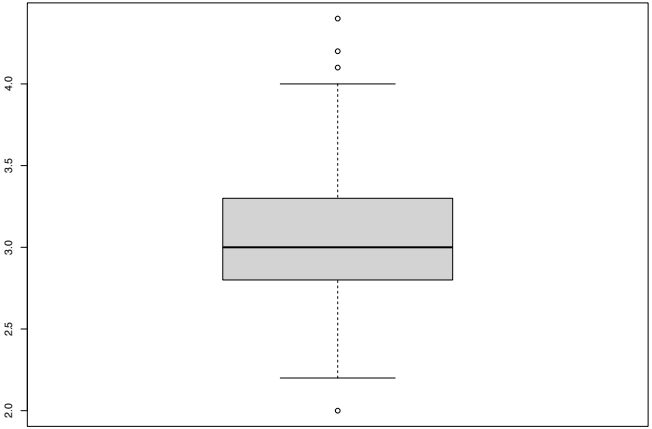
- A box-plot (also known as a box and whisker plot) is another visual representation of spread and skewness of data.
- They are created using the quartiles (Q_1, Q_2, Q_3) and the Inter-quartile range (IQR).
  $IQR = Q_3 - Q_1$
- The whiskers are calculated using:
  lower-whisker: $Q_1 - 1.5 \times IQR$
  upper-whisker: $Q_3 + 1.5 \times IQR$
- The multiplicative factor of 1.5 is used to strike a balance Between rigor and simplicity.

Boxplot Ctd...

Let's make some boxplots using the `Sepal.Width` from the `iris` data set.

```
## get help about the boxplot function
help(boxplot)

## Create a boxplot of sepal widths
boxplot(iris$Sepal.Width)
```

Boxplot Ctd...

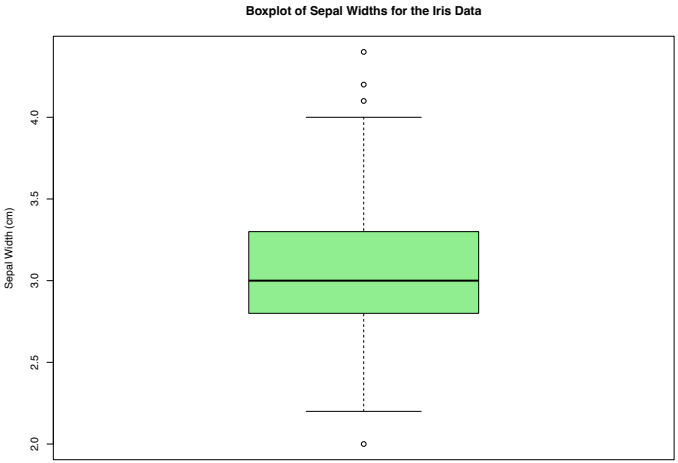


Boxplot Ctd...

We can make the boxplot appear more professional

```
## get help about the boxplot function
help(boxplot)

## Create a boxplot of sepal widths
boxplot(iris$Sepal.Width,
        ylab = "Sepal Width (cm)",
        col = "lightgreen",
        main = "Boxplot of Sepal Widths for the Iris Data")
```

Boxplot Ctd...

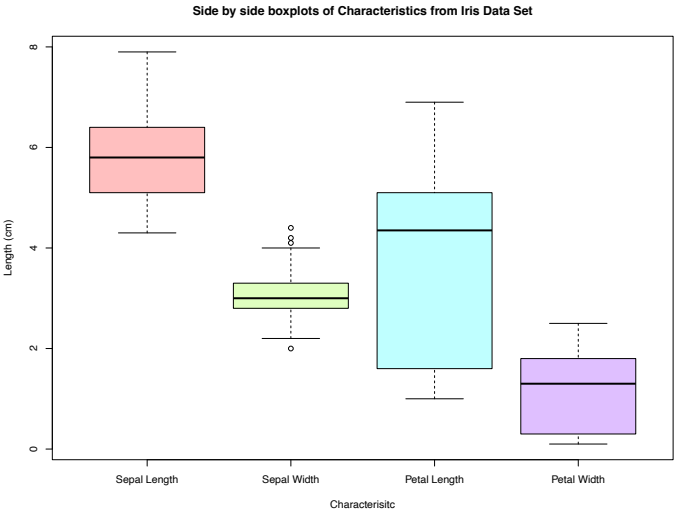

Boxplot of Sepal Widths for the Iris Data
Sepal Width (cm)
2.0
2.5
3.0
3.5
4.0

Boxplot Ctd...

Let's create box-plots for all the (numeric) variables

```
boxplot(iris[,1:4],
        names=c("Sepal Length", "Sepal Width", "Petal Length", "Petal
    Width"),
        xlab = "Characterisitc",
        ylab = "Length (cm)",
        col = adjustcolor(rainbow(4), 0.25),
        main = "Side by side boxplots of Characteristics from Iris Data
     Set")
```

Boxplot Ctd...

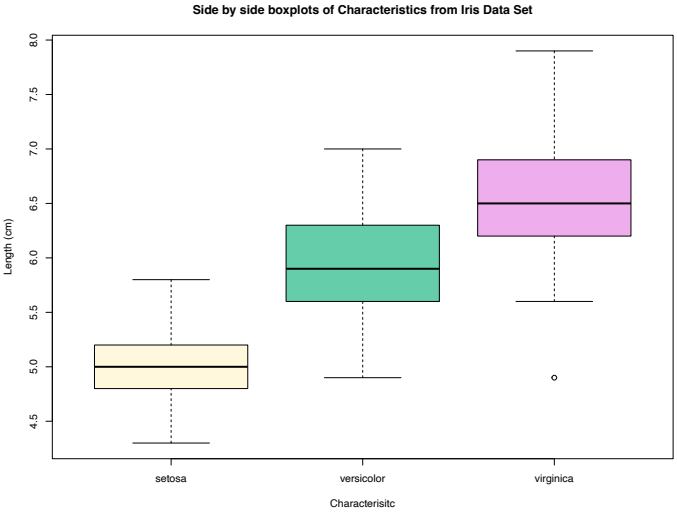

Side by side boxplots of Characteristics from Iris Data Set
Length (cm)
0
2
4
6
8
Sepal Length
Sepal Width
Petal Length
Petal Width
Characterisitc

Boxplot Ctd...

Let's examine the sepal length by species

```
boxplot(iris$Sepal.Length ~ iris$Species,
        xlab = "Characterisitc",
        ylab = "Length (cm)",
        col = c("cornsilk", "mediumaquamarine", "plum2"),
        main = "Side by side boxplots of Characteristics from Iris Data
    Set")
```

Boxplot Ctd...

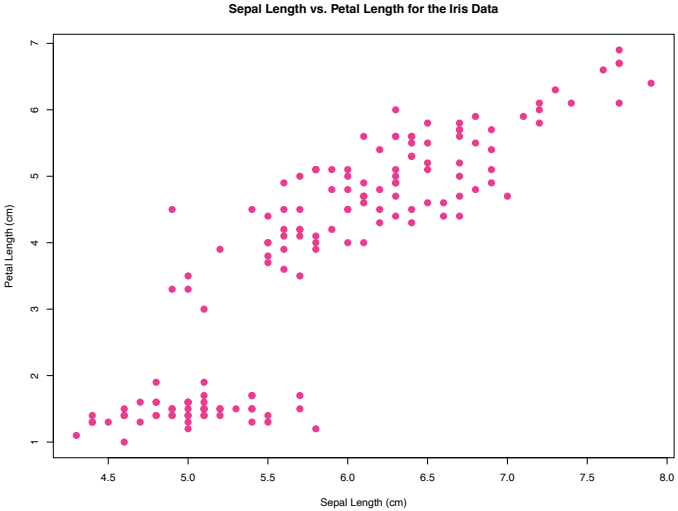

Side by side boxplots of Characteristics from Iris Data Set
Length (cm)
4.5
5.0
5.5
6.0
6.5
7.0
7.5
8.0
setosa
versicolor
virginica
Characterisitc

Scatter Plot

- We can also create scatter plots with our data.
- For example, let's create a scatterplot of Petal Length against Sepal Length.

```
plot(iris$Sepal.Length, iris$Petal.Length,
     xlab = "Sepal Length (cm)",
     ylab = "Petal Length (cm)",
     main = "Sepal Length vs. Petal Length for the Iris Data",
     col = "violetred2",
     pch = 16,
     cex = 1.5)
```

Scatter Plot Ctd...



Outline

1 Operations and Assignment

2 Basic Functions

3 Some Data Structures

4 Plots

5 Libraries

6 Reading a File

Libraries

- A packages is a collection of R functions, data, and code. The directory structure in which a package is are stored is a library.
- R has many libraries which can be used for to perform specific tasks without writing the functionality from scratch.
- We can install packages from the Comprehensive R Archive Network (CRAN) using

```
## Installation
install.packages("package_name")
```

- A package needs to be loaded before it can be used

```
## To load a package
library(package_name)
```

ggplot2

- A very popular library in R is `ggplot2` which is used to create professional graphical summaries and visualizations.
- let's load the package into R.

```
library(ggplot2)
```

- **Note:** If an error message appears similar to the one below

```
Error in library(ggplot2) : there is no package called 'ggplot2'
```

You will need to install the package first

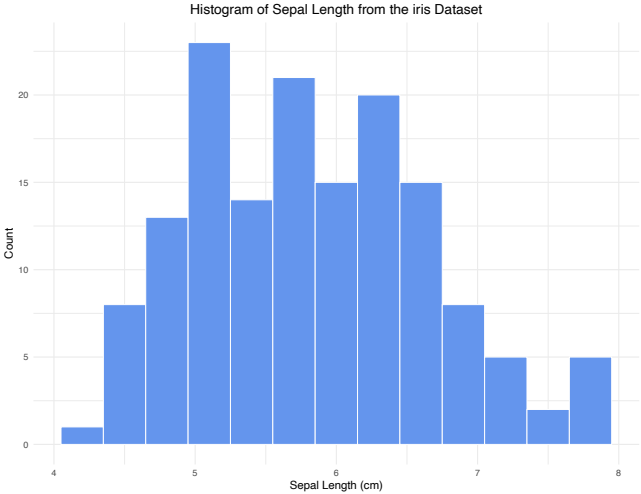
```
install.packages("ggplot2")
## Wait for installation to finish

library(ggplot2)
```

ggplot2 Ctd...

- Let's make a histogram for sepal lengths of the iris data set using ggplot2.

```
library(ggplot2)

ggplot(iris, aes(x = Sepal.Length)) +
  geom_histogram(binwidth = 0.3, fill = "cornflowerblue", color = "
    white") +
  labs(
    x = "Sepal Length (cm)",
    y = "Count",
    title = "Histogram of Sepal Length (iris dataset)"
  ) +
  theme_minimal(base_size = 14) +
  theme(plot.title = element_text(hjust = 0.5))
```

ggplot2 Ctd...


Histogram of Sepal Length from the iris Dataset
Count
0
5
10
15
20
4
5
6
7
8
Sepal Length (cm)

plotly

- Another library which can be used for interactive 2D and 3D plots is `plotly`.
- let's load the package into R.

```
library(ggplot2)
```

- **Note:** If an error message appears similar to the one below

```
Error in library(plotly) : there is no package called 'plotly'
```

You will need to install the package first

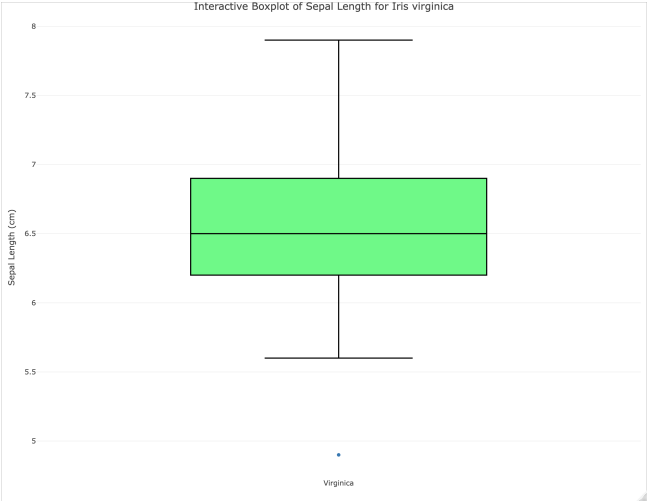
```
install.packages("plotly")
## Wait for installation to finish

library(plotly)
```

plotly Ctd...

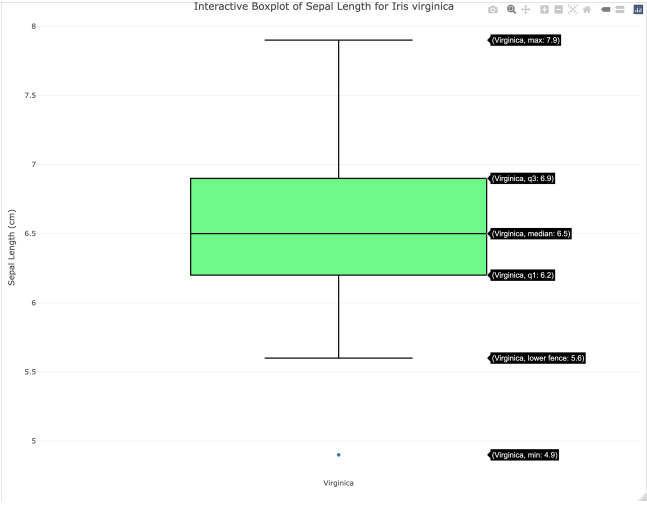
- Let's make a boxplot for sepal lengths of *iris virginica* using `plotly`.

```
library(plotly)

virginica = subset(iris, Species == "virginica")

plot_ly(
  data = virginica,
  y = ~Sepal.Length,
  type = "box",
  name = "Virginica",
  fillcolor = "springgreen",
  line = list(color = "black")
) %>%
  layout(
    title = "Interactive Boxplot of Sepal Length for Iris
   virginica",
    yaxis = list(title = "Sepal Length (cm)")
  )
```

plotly Ctd...


Interactive Boxplot of Sepal Length for Iris virginica
8
7.5
7
6.5
6
5.5
5
Sepal Length (cm)
Virginica

plotly Ctd...


Interactive Boxplot of Sepal Length for Iris virginica
8
7.5
7
6.5
6
5.5
5
Sepal Length (cm)
(Virginica, max: 7.9)
(Virginica, q3: 6.9)
(Virginica, median: 6.5)
(Virginica, q1: 6.2)
(Virginica, lower fence: 5.6)
(Virginica, min: 4.9)
Virginica

Libraries

- There are many other libraries and packages which exist and we only examined `ggplot2` and `plotly` to provide a sample.
- There are many other packages available in R for for manipulating data, creating visualizations, reading various file formats, analyzing statistical models, machine learning, creating interactive apps and dashboards, web-scraping, connecting to databases, working with maps and geospatial data etc.

Outline

1 Operations and Assignment

2 Basic Functions

3 Some Data Structures

4 Plots

5 Libraries

6 Reading a File

Reading a File

- There are several ways to read various file into R.
- We will examine *some* of the ways to read a `.csv` file into R.
- One way is to read the file using the `read.csv()` function directly from the file path.

```
object_name = read.csv("~/path_to_file/file_name.csv")
```

- The file can also be read by first setting the working directory.

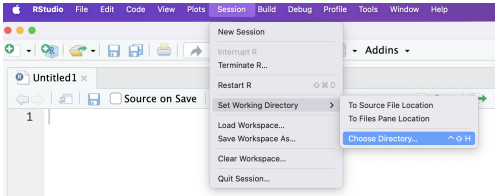
```
setwd("~/path_to_file/")

object_name = read.csv("file_name.csv")
```

- Another way is to use the `file.choose()` function.

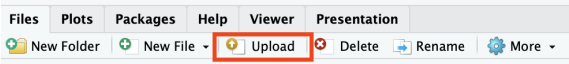
```
object_name = read.csv(file.choose())
```

Reading a File Ctd...

- In RStudio, the working diretcory can be set from the menu bar:
  Session → Set Working Directory → Choose Directory



- Using RStudio on the Jupyterhub, we can use the *Upload* button.



Reading a File Ctd...

- Download the *Canada_Diabetes_2022.csv* file from Quercus.

Column	Description
REF_DATE	The reference year of the data
GEO	The geographic location (province or territory of Canada).
DGUID	The Data GUID (unique identifier) used by Statistics Canada for linking to metadata and geographic boundaries.
Age group	The age category of the population covered (in this dataset: *Total, 18 years and over*).
Sex	The sex breakdown (in this dataset: *Both sexes*).
Indicators	The health indicator being measured (here: *Diabetes*).
VALUE	The numeric value of the prevalence percentage
Characteristics	The unit or type of measure (here: *Percent*).

Table 1: Explanation of the columns in `Canada_Diabetes_2022.csv`. Data is from the Statistics Canada. Table 13-10-0096-01 Health characteristics, annual estimates, inactive

Reading a File

- Read the file into an object called *diabetes* using any of the methods described earlier, such as

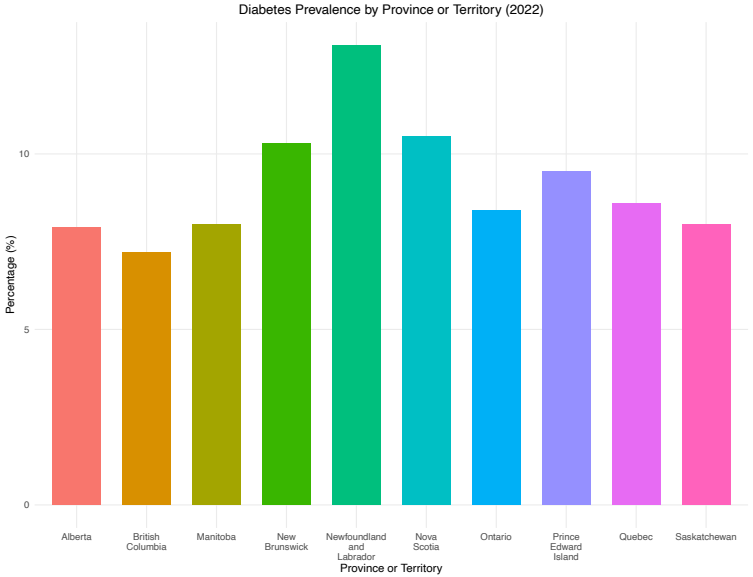
```
diabetes = read.csv("~/path_to_file/Canada_Diabetes_2022.csv",
    stringsAsFactors = FALSE)
```

- We can check that the file has been read correctly

```
diabetes
```

- Let's use the object to create a barplot and an interactive map.

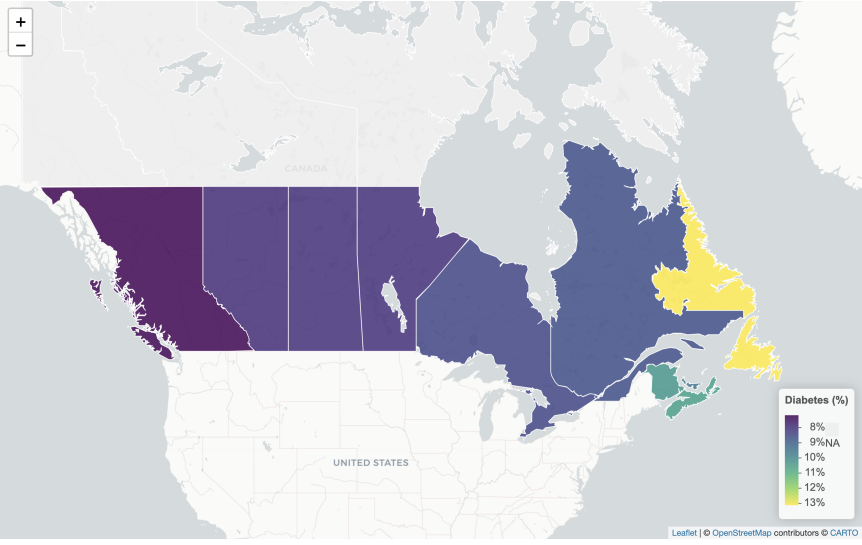
```r
library(ggplot2)
library(stringr)

diabetes = read.csv("~/path_to_file/Canada_Diabetes_2022.csv",
    stringsAsFactors = FALSE)

ggplot(diabetes, aes(x = GEO, y = VALUE, fill = GEO)) +
  geom_col(width = 0.7, show.legend = FALSE) +
  scale_x_discrete(labels = function(x) str_wrap(x, width = 8)) +  #
    wrap long names
  labs(
    title = "Diabetes Prevalence by Province or Territory (2022)",
    x = "Province or Territory",
    y = "Percentage (%)"
  ) +
  theme_minimal(base_size = 12) +
  theme(
    panel.grid.minor = element_blank(),
    plot.title = element_text(hjust = 0.5)
  )
```


Diabetes Prevalence by Province or Territory (2022)
Percentage (%)
0
5
10
Alberta
British Columbia
Manitoba
New Brunswick
Newfoundland and Labrador
Nova Scotia
Ontario
Prince Edward Island
Quebec
Saskatchewan
Province or Territory

```r
## install packages if necessary
library(sf)
library(leaflet)
library(rnaturalearth)
library(dplyr)

## Read your data
dat = read.csv("~/path_to_file/Canada_Diabetes_2022.csv",
    stringsAsFactors = FALSE)

## Get province/territory polygons from the sf library
canada = ne_states(country = "canada", returnclass = "sf") %>%
  select(name_en, geometry)

## More code on next slide
```

```r
## Join by name (name_en and GEO)
map_dat = canada %>%
  left_join(dat, by = c("name_en" = "GEO"))

## Color scale for percentages using the viridis palette
pal <- colorNumeric(
  palette = "viridis",
  domain = map_dat$VALUE,
  na.color = "#EEEEEE"
)

## Labels for hover (HTML)
lab = sprintf(
  "<strong>%s</strong><br/>Diabetes: %s%%",
  map_dat$name_en, ifelse(is.na(map_dat$VALUE), "NA", format(round(map_
    dat$VALUE, 1), nsmall = 1))
) %>% lapply(htmltools::HTML)
```

```r
## Interactive map
leaflet(map_dat, options = leafletOptions(minZoom = 2)) |>
  addProviderTiles(providers$CartoDB.Positron) |>
  addPolygons(
    fillColor = ~pal(VALUE),
    color = "white", weight = 1, opacity = 1,
    fillOpacity = 0.85,
    label = lab,
    highlightOptions = highlightOptions(weight = 2, color = "#333333",
   fillOpacity = 0.95, bringToFront = TRUE)
  ) %>%
  addLegend(
    pal = pal, values = ~VALUE, opacity = 0.85,
    title = "Diabetes (%)",
    position = "bottomright",
    labFormat = leaflet::labelFormat(suffix = "%", digits = 1)
  )
```


CANADA
UNITED STATES
Diabetes (%)
8%
9%NA
10%
11%
12%
13%
Leaflet | © OpenStreetMap contributors © CARTO